

Miniprojekt A

DEL 1

Navn: Asger Grundtdal Stidsen

AU-ID: au798909

Afleveringsdato: 13. september 2026

Introduktion

Dette miniprojekt omhandler første del af kurset Digital Signalbehandling, hvor der er udleveret tre lydfiler, Signal 1, Signal 2 og Signal 3. Formålet med projektet er at analysere og behandle lydsignalerne med udgangspunkt i den teori, der er gennemgået i første del af kurset.

Projektet består af i alt 10 opgaver, hvor de tre signaler anvendes til forskellige former for analyse og signalbehandling. Opgaverne omhandler blandt andet forskellige aspekter af sampling, signal- og frekvensanalyse samt behandling af lydsignaler. Den relevante teori vil blive anvendt til at forklare de udførte metoder og resultater.

Indhold

Introduktion	1
0 Global Code	1
1 Opgave: Find antal samples N for alle tre signaler	2
2 Opgave: Plot alle tre signaler med korrekte tidsakser	3
3 Opgave: Find for alle tre signaler: max, min, gennemsnit (mean), rms og effekt . .	7
4 Opgave: Find de tre signalers crest-faktorer, - sammenlign og kommenter	9
5 Opgave: Nedsampel s1 med en faktor 4. Plot – og sammenlign med originalen . .	10
6 Opgave: Lyt – og beskriv forskellen mellem originalen og den nedsamplede version	13
7 Opgave: Sammenlign de to stereo-kanaler i s2 – beskriv forskelle og ligheder . . .	14
8 Opgave: Kvantisering af s2 med 4 bit	17
9 Opgave: Fade-out på s3 med eksponentielt aftagende envelope	21
10 Opgave 10: Lav en ny udgave af s3, hvor der er et enkelt ekko med et delay på 100ms.	24

0 Global Code

Opgaven er lavet i Python. Nedenfor ses de biblioteker, der bruges gennem hele rapporten, samt indlæsning af de tre lydsignaler.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import sympy as sym
4 import scipy as sci
5 import pandas as pd
6 import soundfile as sf
7 from IPython.display import Audio, display
8
9
10 # Load audio files
11 audio_s1, fs_s1 = sf.read("./Signal_s1.mp3")
12 audio_s2, fs_s2 = sf.read("./Signal_s2.mp3")
13 audio_s3, fs_s3 = sf.read("./Signal_s3.mp3")
14
15 f_s = 44100
16 t_s = 1/f_s
```

1 Opgave: Find antal samples N for alle tre signaler

1.1 Forberedelse

Når vi loader lydfilen ind i Python, har den dimensionen (N : 2). Det betyder, at den indeholder (N) samples, hvor hver sample består af en venstre og en højre lydkanal. Målet er at finde antallet af samples. Dette kan gøres med funktionen `len(x)` med x værende den loaded lyd.

1.2 Kode

```
1 N_s1 = len(audio_s1)
2 N_s2 = len(audio_s2)
3 N_s3 = len(audio_s3)
4
5
6 print(f"Der er {N_s1} samples i s1")
7 print(f"Der er {N_s2} samples i s2")
8 print(f"Der er {N_s3} samples i s3")
```

1.3 Resultat

Kørsel af koden gav følgende antal samples:

```
1 Der er 4586112 samples i s1
2 Der er 9527040 samples i s2
3 Der er 1382400 samples i s3
```

1.4 Diskussion og konklusion

Her kan vi se, at der er flest samples i signal 2, hvilket også stemmer overens med lydfilens længde på tre minutter, hvor de to andre signaler er på henholdsvis et et halvt minut og et halvt minut.

2 Opgave: Plot alle tre signaler med korrekte tidsakser

2.1 Opgavebeskrivelse

Plot alle tre signaler med korrekte tidsakser, og sæt 'xlabel', 'ylabel' og 'title' på

2.2 Forberedelse

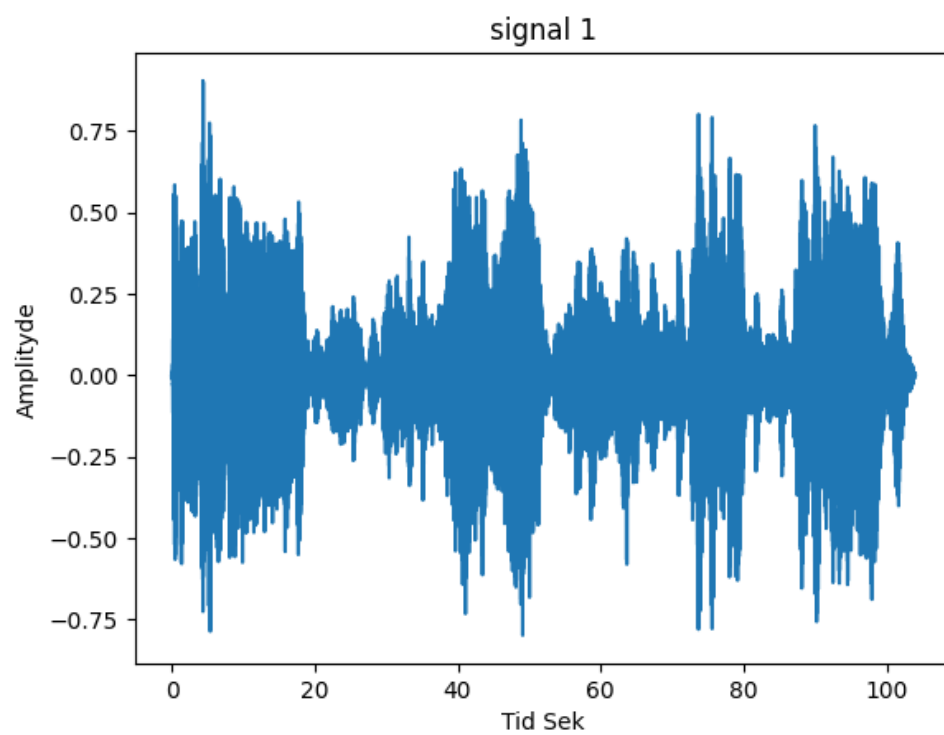
Hvert sample i signalet har ikke en fastlagt tid. Ved at plotte samples direkte vil den korrekte tidsakse derfor ikke være repræsenteret. Derfor skal der oprettes en tidsakse, som angiver, på hvilket tidspunkt hvert sample er blevet samlet. Dette gøres ved hjælp af et index-array, som multipliceres med tiden mellem hvert sample.

Det er også her vigtigt at pointere, at signal 2 er stereo. Så den har to separate kanaler med forskellig lyddata. Dermed skal hver kanal plottes for at få alt dataen plottet.

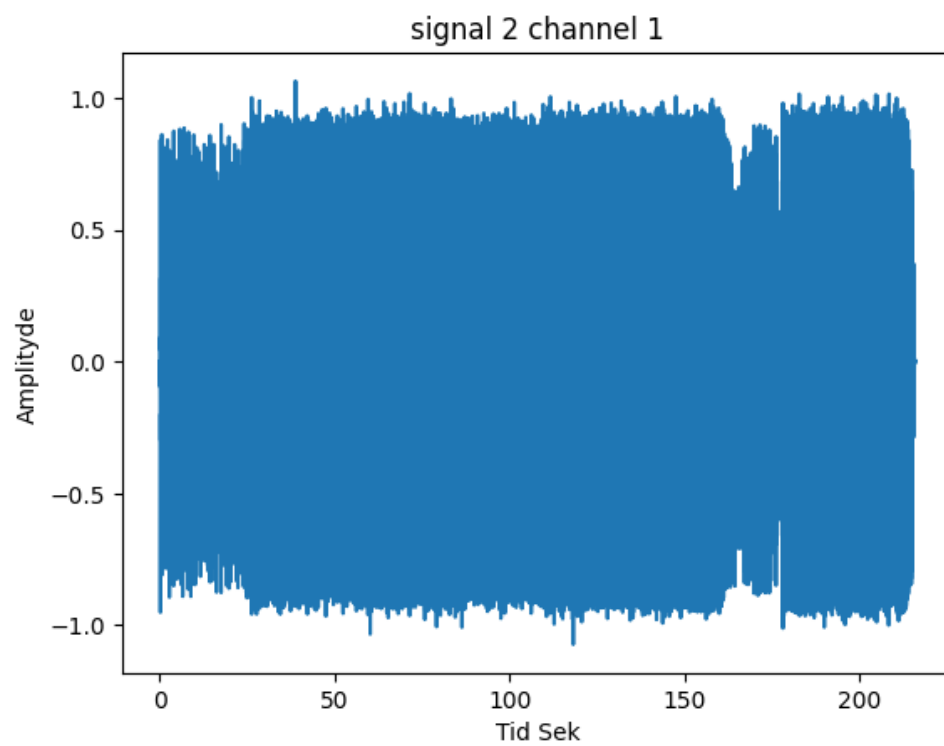
2.3 Kode

```
1 #get tid for hver sample
2 t_s1 = np.arange(N_s1)*t_s
3 t_s2 = np.arange(N_s2)*t_s
4 t_s3 = np.arange(N_s3)*t_s
5
6 #function til at plotte et et lyd array
7 def plotAudio(title_,t,a):
8     plt.plot(t,a)
9     plt.xlabel("Tid Sek")
10    plt.ylabel("Amplityde")
11    plt.title(title_)
12    plt.show()
13
14 # plot de tre signaler
15 plotAudio("signal 1",t_s1,audio_s1[:,0])
16 plotAudio("signal 2 channel 1",t_s2,audio_s2[:,0])
17 plotAudio("signal 2 channel 2",t_s2,audio_s2[:,1])
18 plotAudio("signal 3",t_s3,audio_s3[:,0])
```

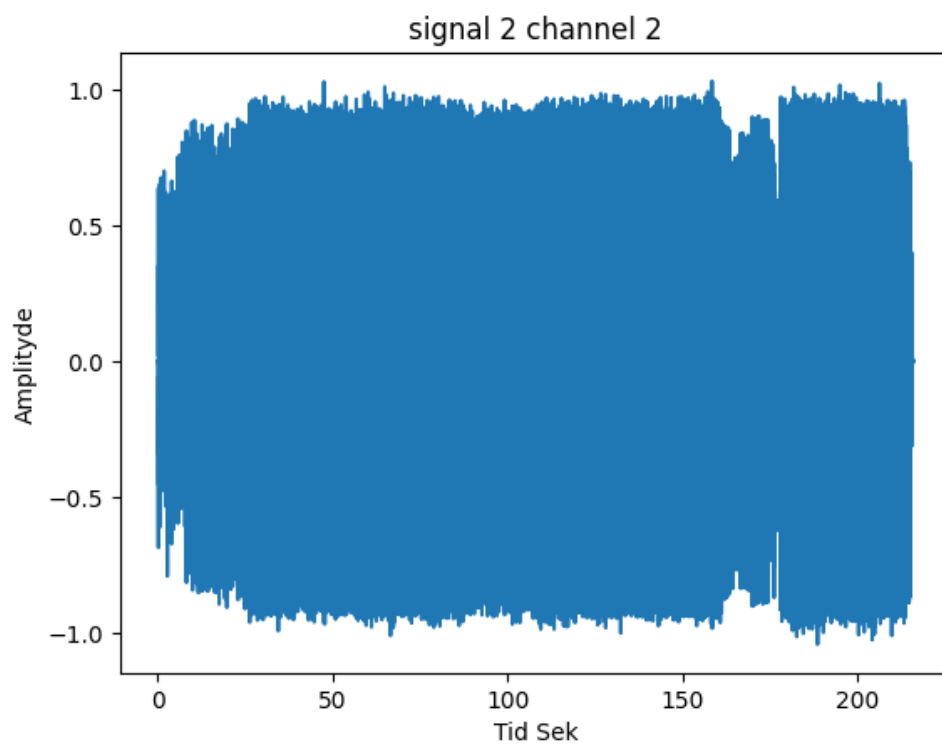
2.4 Resultat



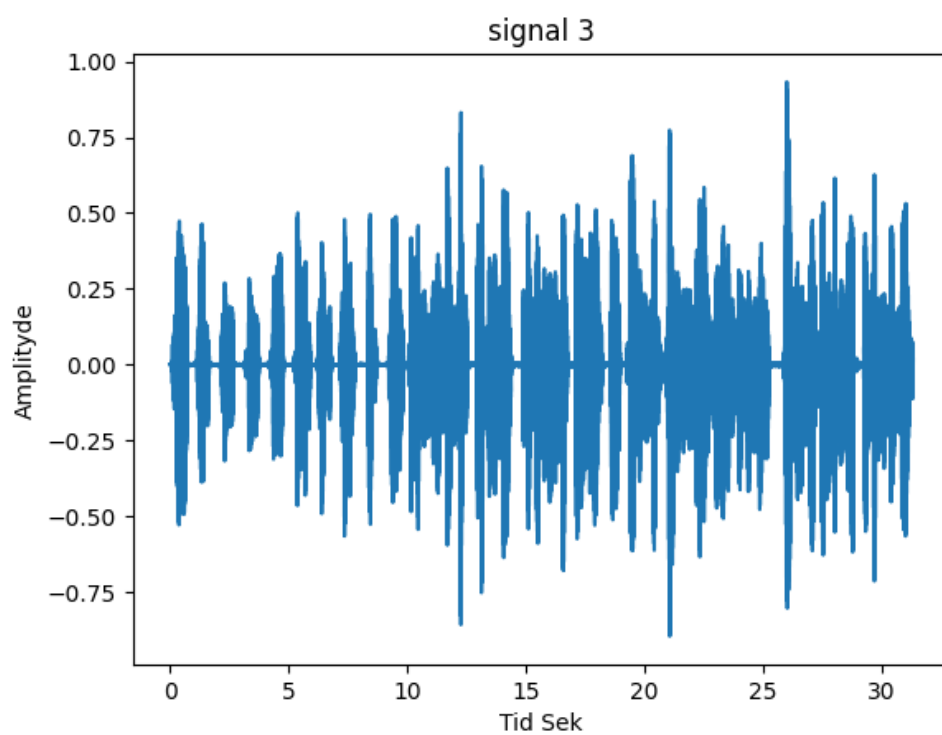
Figur 1: Plot af signal 1



Figur 2: Plot af signal 2 kanal 1



Figur 3: Plot af signal 2 kanal 2



Figur 4: Plot af signal 3

2.5 Diskussion og konklusion

vi kan se at signalerne er meget forskellige fra hinanden. Signal 2 på Figur 2 har meget tætte og jævne amplitudeudsving, mens signal 3 på Figur 4 har større mellemrum mellem amplitudeudsvingene. Signal 1 på Figur 1 ligger mellem de to, hvor amplitudeudsvingene er mindre tætte end i signal 2, men tættere end i signal 3.

Derudover kan vi se, at signal 2 ikke er ens på begge kanaler. Dette stemmer overens med, at signal 2 er et stereosignal og derfor indeholder en venstre og en højre kanal, som ikke nødvendigvis har samme amplituder. Forskellen mellem kanalerne ses ved sammenligning af Figur 2 og Figur 3.

3 Opgave: Find for alle tre signaler: max, min, gennemsnit (mean), rms og effekt

3.1 Forberedelse

Ved benyttelse af biblioteket NumPy er mange af disse værdier, der skal findes, allerede en indbygget funktion. Her vil der kun blive implementeret en funktion til at finde RMS-værdien og effekten. Alt dette er forholdsvis lige til og kræver ikke yderligere forklaring. Dog er det vigtigt at pointere, at signal 2 er stereo, og dermed er det vigtigt at inkludere hele signalet. Derfor vil kanal 2 blive lagt bagefter kanal 1 for at lave ét langt lydsignal, som vil give maks., min., median, RMS og effekten over alt dataen.

3.2 Kode

```
1 #max
2 max_s1 = np.max(audio_s1[:,0])
3 max_s2 = np.max([audio_s2[:, 0], audio_s2[:, 1]])
4 max_s3 = np.max(audio_s3[:,0])
5
6 #min
7 min_s1 = np.min(audio_s1[:,0])
8 min_s2 = np.min([audio_s2[:, 0], audio_s2[:, 1]])
9 min_s3 = np.min(audio_s3[:,0])
10
11 #mean
12 mean_s1 = np.mean(audio_s1[:,0])
13 mean_s2 = np.mean([audio_s2[:, 0], audio_s2[:, 1]])
14 mean_s3 = np.mean(audio_s3[:,0])
15
16 #rms
17 def rms(signal):
18     s = np.pow(signal,2)
19     ms = np.mean(s)
20     rms = np.sqrt(ms)
21     return rms
22
23 rms_s1 = rms(audio_s1[:,0])
24 rms_s2 = rms([audio_s2[:, 0], audio_s2[:, 1]])
25 rms_s3 = rms(audio_s3[:,0])
26
27
28 #effekt
29 def e(signal):
30     s = np.pow(signal,2)
31     ms = np.mean(s)
32     return ms
33
34 e_s1 = e(audio_s1[:,0])
35 e_s2 = e([audio_s2[:, 0], audio_s2[:, 1]])
36 e_s3 = e(audio_s3[:,0])
37
38 print(f"signal 1:")
39 print(f"max: {max_s1:.3f}")
40 print(f"min: {min_s1:.3f}")
41 print(f"mean: {mean_s1:.3e}")
```

```

42 print(f"rms: {rms_s1:.3f}")
43 print(f"effekt: {e_s1:.3e}")
44 print()
45 print(f"signal 2:")
46 print(f"max: {max_s2:.3f}")
47 print(f"min: {min_s2:.3f}")
48 print(f"mean: {mean_s2:.3e}")
49 print(f"rms: {rms_s2:.3f}")
50 print(f"effekt: {e_s2:.3e}")
51 print()
52 print(f"signal 3:")
53 print(f"max: {max_s3:.3f}")
54 print(f"min: {min_s3:.3f}")
55 print(f"mean: {mean_s3:.3e}")
56 print(f"rms: {rms_s3:.3f}")
57 print(f"effekt: {e_s3:.3e}")
58 print()

```

3.3 Resultat

```

1  signal 1:
2  max: 0.903
3  min: -0.798
4  mean: -4.102e-05
5  rms: 0.116
6  effekt: 1.352e-02
7
8  signal 2:
9  max: 1.064
10 min: -1.073
11 mean: -1.032e-05
12 rms: 0.337
13 effekt: 1.133e-01
14
15 signal 3:
16 max: 0.933
17 min: -0.897
18 mean: -5.575e-04
19 rms: 0.089
20 effekt: 7.955e-03

```

3.4 Diskussion og konklusion

Ud fra resultatet kan det observeres, at der er større effekt og rms i signal 2 end i signal 1 og 3. Dette stemmer også overens med det forventede. Vi kan se i Figur 2, at signalet har en større amplitude over en større del af tiden end fx. signalet i Figur 1, hvor signalet ikke er lige så tæt og højt i amplituden.

4 Opgave: Find de tre signalers crest-faktorer, - sammenlign og kommenter

4.1 Forberedelse

Crest factor beregnes som forholdet mellem signalets peak-værdi og RMS-værdi ved hjælp af formelen $20 \log_{10} \left(\frac{x_{\text{peak}}}{x_{\text{RMS}}} \right)$. Peak-værdien er den maksimale absolutte amplitude i signalet. Derfor vil der først blive beregnet peak-værdien for de tre signaler, hvorefter disse værdier sammen med RMS-værdierne, fra opgave 3, kan indsættes i formlen for crest factor.

4.2 Kode

```
1 #function til at beregne crest factor
2 def crest_factor(peak,rms):
3     return 20*np.log10(peak/rms)
4
5 #beregne peak
6 peak_s1 = np.max([max_s1,-min_s1])
7 peak_s2 = np.max([max_s2,-min_s2])
8 peak_s3 = np.max([max_s3,-min_s3])
9
10 #beregne crest factor
11 c_s1 = crest_factor(peak_s1,rms_s1)
12 c_s2 = crest_factor(peak_s2,rms_s2)
13 c_s3 = crest_factor(peak_s3,rms_s3)
14
15 #print resultat
16 print("crest_factor:")
17 print(f"signal 1: {c_s1:.2f}")
18 print(f"signal 2: {c_s2:.2f}")
19 print(f"signal 3: {c_s3:.2f}")
```

4.3 Resultat

```
1 crest_factor:
2 signal 1: 17.81
3 signal 2: 10.07
4 signal 3: 20.40
```

4.4 Diskussion og konklusion

Resultatet viser, at signal 2 har den laveste crest factor på 10,07, efterfulgt af signal 1 på 17,81, mens signal 3 har den højeste crest factor på 20,40. Det stemmer godt overens med observationerne fra de tre figurer. I signal 2 i Figur 2 kan der observeres en forholdsvis jævn og høj amplitude uden tydelige amplitudespidser, hvilket resulterer i en lavere crest factor. Figur 4 der viser Signal 3 er derimod opbyggede af mange korte og tydelige amplitudespidser, hvilket giver en større forskel mellem peak og RMS og dermed en højere crest factor. Signal 1 på i Figur 1 ligger mellem de to signaler, hvor der kan observeres bredere amplitudespidser, men uden de samme korte og markante spidser som i signal 3. Dette stemmer godt overens med de beregnede crest factor-værdier for de tre signaler.

5 Opgave: Nedsampel s1 med en faktor 4. Plot – og sammenlign med originalen

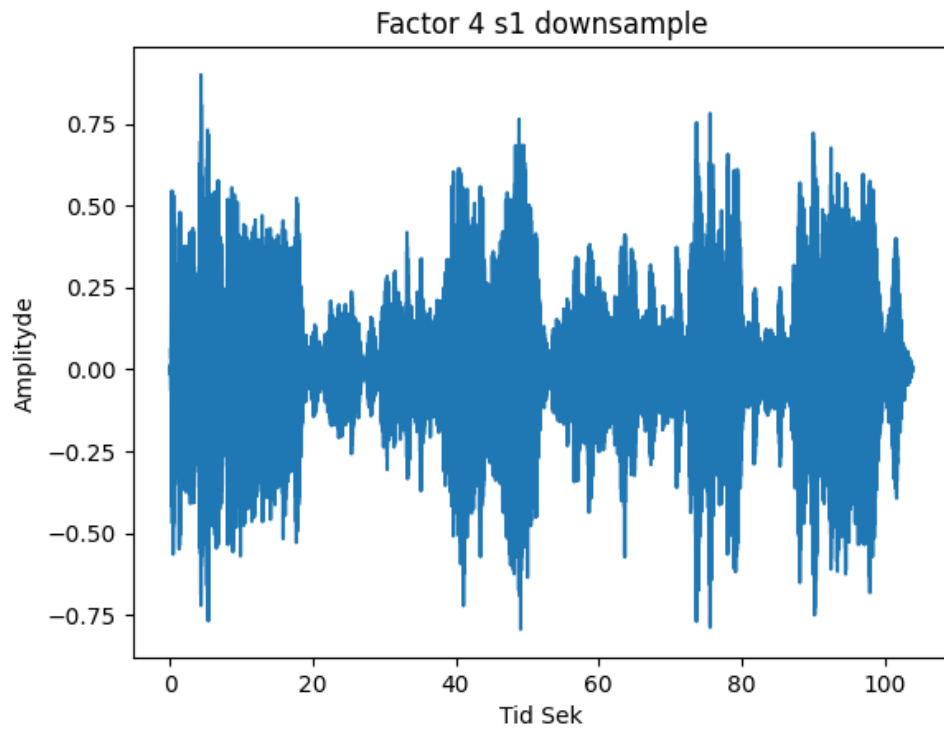
5.1 Forberedelse

Til denne opgave, hvor signalet skal downsamplers med en faktor på 4, vil funktionen `decimate` blive anvendt. Funktionen bruger et filter før downsampling, hvilket fjerner aliasing. Aliasing kan opstå ved downsampling, hvor højere frekvenser bliver spejlet ned i det nye frekvensområde. Derudover er det vigtigt at beregne den nye samplingstiden, så tidsaksen forbliver korrekt efter downsampling og dermed ikke ændres.

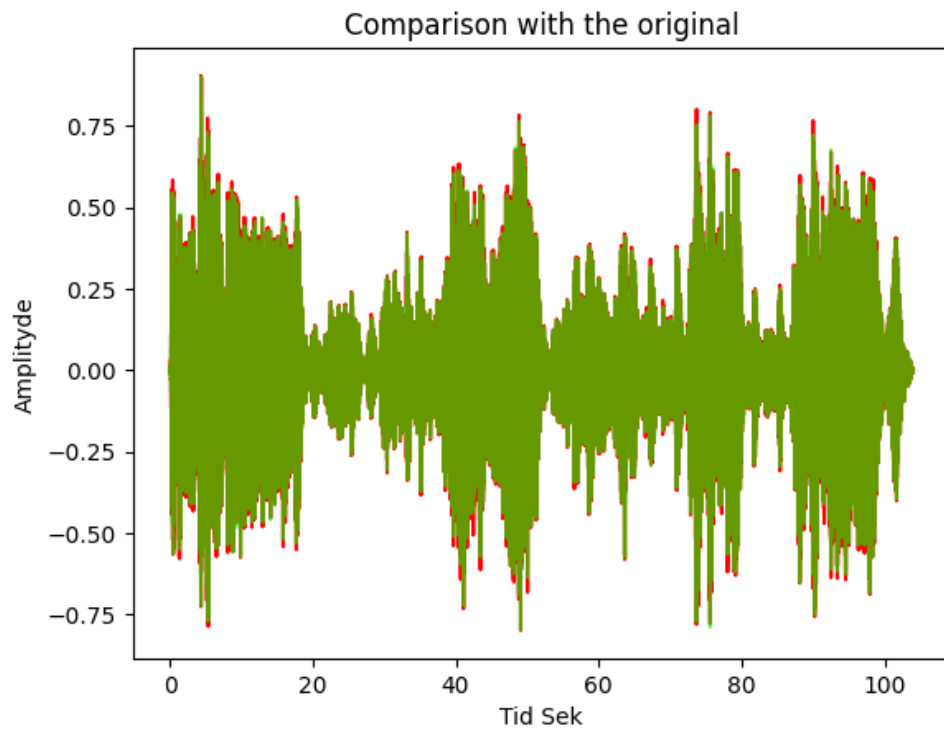
5.2 Kode

```
1 #factor
2 factor = 4
3
4 downsampled_f4_s1 = sci.signal.decimate(audio_s1[:,0], factor)
5
6 #tid mellem hver sample
7 downsampled_f4_t_s = t_s*4
8
9 #lav den nye tidsakse til det downsampled signal
10 downsampled_f4_t_s1 = np.arange(len(downsampled_f4_s1))*downsampled_f4_t_s
11
12 #plot det nye signal
13 plotAudio("Factor 4 s1 downsample",downsampled_f4_t_s1,downsampled_f4_s1)
14
15 #sammenlign de to signaler i samme plot
16 plt.plot(t_s1, audio_s1[:,0], c="red")
17 plt.plot(downsampled_f4_t_s1, downsampled_f4_s1, c="#00FF0099")
18 plt.xlabel("Tid Sek")
19 plt.ylabel("Amplitude")
20 plt.title("Comparison with the original")
21 plt.show()
```

5.3 Resultat



Figur 5: Plot af signal 1 downsamplet



Figur 6: Plot af signal 1 sammenligning med downsample

5.4 Diskussion og konklusion

På Figur 5 kan der ikke observeres nogen tydelig ændring i signalet på plottet. Ved sammenligning med det originale signal på Figur 6 kan der dog ses en mindre ændring, særligt ved de yderste amplitudespidser, hvor der kan observeres et væsentligt tab. Dette kan forklares ud fra teorien bag downsampling, da høje frekvenser kan forårsage aliasing, hvis de ikke fjernes inden downsampling.

Ved downsampling af et signal kan høje frekvenser forårsage aliasing, hvilket resulterer i uønskede udsving i det lavere frekvensområde. Dette hænger sammen med Shannons samplingssætning, som angiver, at frekvenser over halvdelen af samplingsfrekvensen, altså Nyquist-frekvensen, ikke kan repræsenteres korrekt og derfor vil blive foldet ned i det lavere frekvensområde.

For at undgå dette anvender funktionen ‘decimate’ et lavpasfilter før downsampling. Filteret dæmper de frekvenser, som ligger over den nye Nyquist-frekvens, så de ikke efterfølgende skaber aliasing i det lavere frekvensområde. Det observerede tab ved de yderste amplitudespidser kan derfor blandt andet skyldes, at højere frekvenser er blevet dæmpet af lavpasfilteret inden downsampling.

6 Opgave: Lyt – og beskriv forskellen mellem originalen og den nedsamplede version

6.1 Forberedelse

Her vil et biblioteket blive brugt til at afspille dataen som lydfil. Resultatet ligger i bilag.

6.2 Kode

```
1 #udskriv de to lidsamples så der kan høres forskel
2 display(Audio(audio_s1[:,0], rate=f_s))
3 display(Audio(downsampled_f4_s1, rate=f_s//factor))
```

6.3 Resultat

Bilag "Signal_s1_.mp3"

Bilag "Signal_s1_downsampled.wav"

6.4 Diskussion og konklusion

Lyden i den downsamplede version er mere dæmpet, og det føles, som om man befinder sig længere væk fra musikken i stedet for at være til stede i samme rum. Den originale version lyder mere nærværende, mens den downsamplede version virker mere fjern.

Dette stemmer også overens med det forventede ved downsampling, da de højeste frekvenser går tabt. Høje frekvenser bliver generelt absorberet mere af luften end lave frekvenser, når lyden bevæger sig over længere afstande. Da disse frekvenser allerede er fjernet ved downsampling, kan det bidrage til oplevelsen af, at lyden kommer fra længere væk, som om man ikke befinder sig i samme rum som musikken.

7 Opgave: Sammenlign de to stereo-kanaler i s2 – beskriv forskelle og ligheder

7.1 Forberedelse

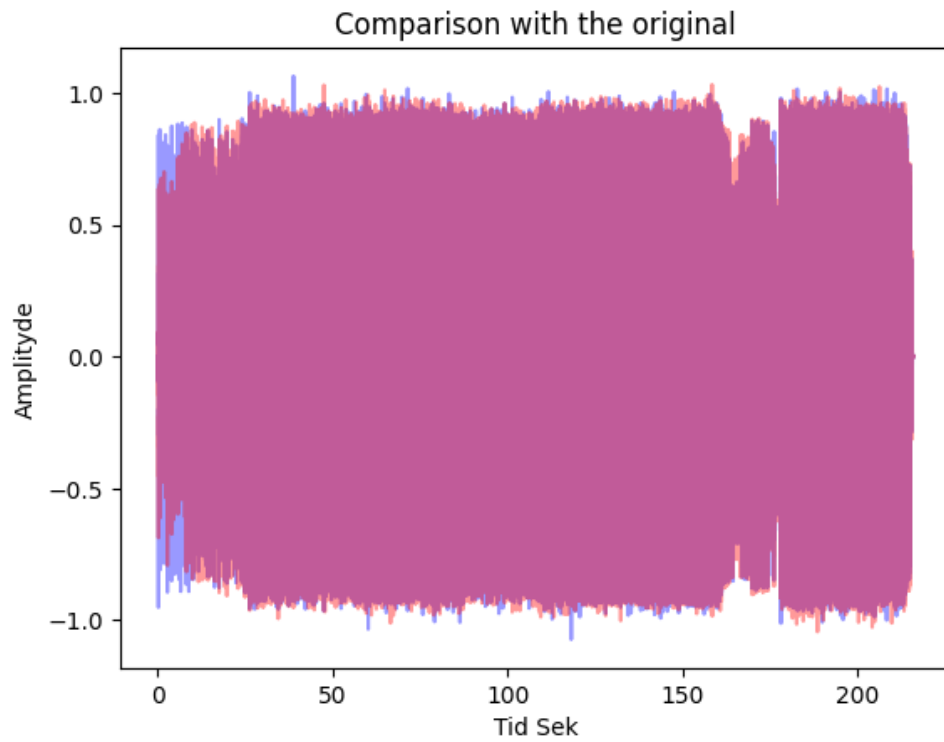
Da jeg ikke selv kan høre nogen forskel, vil jeg plotte de to signaler i samme graf. Her vil det venstre signal være blåt, mens det højre signal vil være rødt. Der vil blive plottet både de to lyd-signaler og deres frekvensspektre for at identificere eventuelle direkte forskelle og frekvensstendenser mellem de to signaler.

Det er vigtigt ved visning af frekvensspektret at følge Shannons samplingssætning og derfor kun plotte frekvenser op til halvdelen af samplingsfrekvensen. Denne grænse kaldes Nyquist-frekvensen. Frekvenserne over Nyquist-frekvensen indeholder ikke yderligere information om det originale signal, da den anden halvdel af spektret er en spejling af den første halvdel.

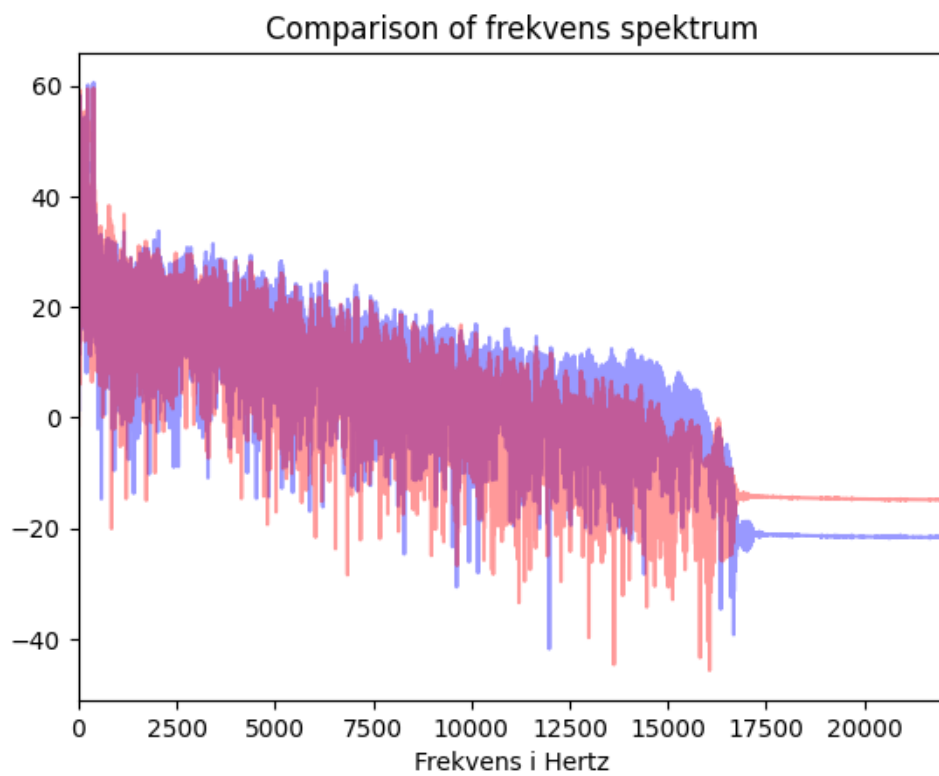
7.2 Kode

```
1 #plot signals 2's kanaler for sammenligning
2 plt.plot(t_s2, audio_s2[:,0], c="#0000ff66")
3 plt.plot(t_s2, audio_s2[:,1], c="#ff000066")
4 plt.xlabel("Tid Sek")
5 plt.ylabel("Amplitude")
6 plt.title("Comparison with the original")
7 plt.show()
8
9
10
11 #plot signals 2's kanalers frekvens spektrum for sammenligning
12 S2_left = np.fft.fft(audio_s2[:,0], int(f_s))
13 S2_right = np.fft.fft(audio_s2[:,1], int(f_s))
14
15 #plot
16 plt.figure(4)
17 plt.clf()
18 plt.plot(20 * np.log10(np.abs(S2_left)), c="#0000ff66")
19 plt.plot(20 * np.log10(np.abs(S2_right)), c="#ff000066")
20 plt.xlim(0, f_s/2)
21 plt.xlabel("Frekvens i Hertz")
22 plt.title("Comparison of frekvens spektrum")
23 plt.show()
```

7.3 Resultat



Figur 7: Plot af signal 2 sammenligning af kanaler



Figur 8: Plot af signal 2 sammenligning af kanaler frekvens spektrum

7.4 Diskussion og konklusion

Ved en direkte sammenligning af lyden er der ikke nogen stor forskel mellem de to kanaler, hvilket forklarer hvorfor jeg ikke kan høre forskel. Den tydeligste forskel ses i starten, hvor den højre kanal har en højere amplitude end den venstre kanal. Hvis vi ser på frekvensspektret for de to kanaler, kan vi også se, at de hovedsageligt indeholder de samme frekvenser. Ved en relativ sammenligning af frekvensspektrene kan det desuden ses, at den højre kanal indeholder flere høje frekvenser end den venstre kanal, særligt i området fra omkring 10 kHz til 15 kHz.

8 Opgave: Kvantisering af s2 med 4 bit

8.1 Opgavebeskrivelse

Kvantiser s2 (venstre kanal) med 4 bit (brug egen funktion eller den på BS). Plot og lyt, - sammenlign med originalen og kommenter

8.2 Forberedelse

Til denne opgave vil jeg lave min egen kvantificeringsfunktion, som sigter efter at bevare amplituden så præcis som muligt. Dette inkluderer at bevare signalets maksimale og minimale amplitude og kvantificere signalet i 4 bit eller 16 niveauer mellem den maksimale og minimale amplitude.

For at udføre dette vil funktionen først finde antallet af steps, eller niveauer, ved at beregne 2^{bits} . Derefter vil funktionen skalere signalet, så det ligger mellem 0 og 1. Dette gøres, da signalet ikke nødvendigvis ligger mellem -1 og 1 . Ved at skalere signalet til et kendt interval bliver det muligt at tilpasse samples til de ønskede kvantiseringsniveauer.

Efter signalet er skaleret til intervallet fra 0 til 1, multipliceres signalet med antallet af steps minus 1. Dette skyldes, at værdien 0 også tæller som et step. Ved 4 bit er der derfor $2^4 = 16$ niveauer, som går fra 0 til 15. Samples vil dermed have værdier mellem 0 og 15, som svarer til de 16 forskellige kvantiseringsniveauer.

Herefter afrundes værdierne til heltal, så samples placeres på et af de 16 kvantiseringsniveauer. Derefter skales signalet tilbage til intervallet fra 0 til 1. Til sidst skales signalet tilbage til det oprindelige amplitudinterval mellem den maksimale og minimale amplitude. På den måde bevares signalets oprindelige amplitudområde, mens signalet samtidig bliver kvantiseret.

I koden vil jeg desuden kontrollere, at amplitudeforholdet bevares, og at kvantiseringsprocessen ikke ændrer signalets overordnede amplitudeforhold.

8.3 Kode

```
1 # få maks og min a venstre kanal
2 max_s2_left = np.max(audio_s2[:,0])
3 min_s2_left = np.min(audio_s2[:,0])
4
5 #function til at kvantificere med signalet max min og antal bits
6 def quantize(signal,max,min,bits):
7
8     #få antallet af steps signalet skal beskrives på
9     steps = 2**bits
10
11     #formater signalet til en værdi mellem 1 og 0
12     mapped_signal = (signal - min) / (max - min)
13
14     #gang med antallet af steps for at kunne arrunde til hele tal senere
15     #da 0 er en af stepsne er skal signalet kun forstærkes til værdier mellem 0 til 15
16     amplified_mapped_signal = mapped_signal * (steps-1)
17
18     #afrund til hele tal for at få signalet passet ind på de 16 steps
19     stepped_amplified_mapped_signal = np.round(amplified_mapped_signal)
20
21     #divider signalet ned igen så det bliver et signal mellem 0 og 1
22     stepped_mapped_signal = stepped_amplified_mapped_signal / (steps-1)
```

```

23
24     #formater signalet tilbage til signales originale maks- og minimumsværdi
25     stepped_signal = stepped_maped_signal * (max - min) + min
26
27     return stepped_signal
28
29 #kvanticer signalet
30 q4_s2_left = quantize(audio_s2[:,0],max_s2_left,min_s2_left,4)
31
32 #check to see if amplitude is kepted
33 max_q4_s2_left = np.max(q4_s2_left)
34 min_q4_s2_left = np.min(q4_s2_left)
35
36 #pring afstanden mellem max og min før kvantisering og efter for at se om forholdet æ
    ndre sig eller om der er fejl
37 print(f"delta_max: {np.abs(max_s2_left-max_q4_s2_left):.3e}")
38 print(f"delta_min: {np.abs(min_s2_left-min_q4_s2_left):.3e}")
39
40
41 #plot
42 plotAudio("s2 Kvantiseret til 4bit",t_s2,q4_s2_left)
43
44
45 #plot sammenligning med originalen
46 plt.plot(t_s2,audio_s2[:,0],c="red")
47 plt.plot(t_s2,q4_s2_left,c="#00FF0099")
48 plt.xlabel("Tid Sek")
49 plt.ylabel("Amplityde")
50 plt.title("sammenlign kvantisering med originalen")
51 plt.show()
52
53 display(Audio(q4_s2_left,rate=f_s))

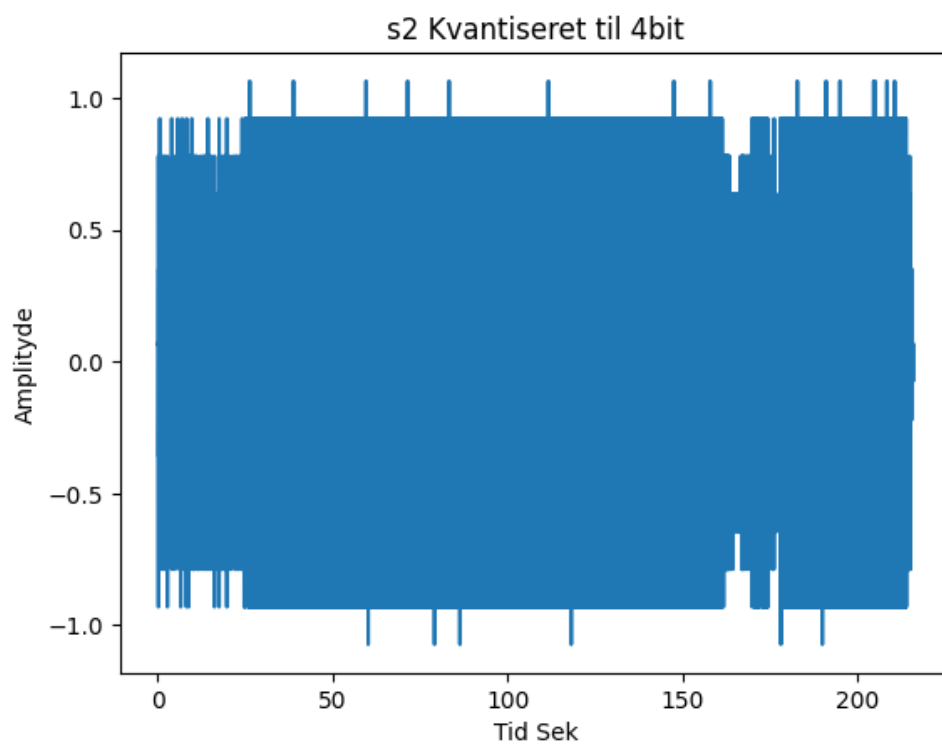
```

8.4 Resultat

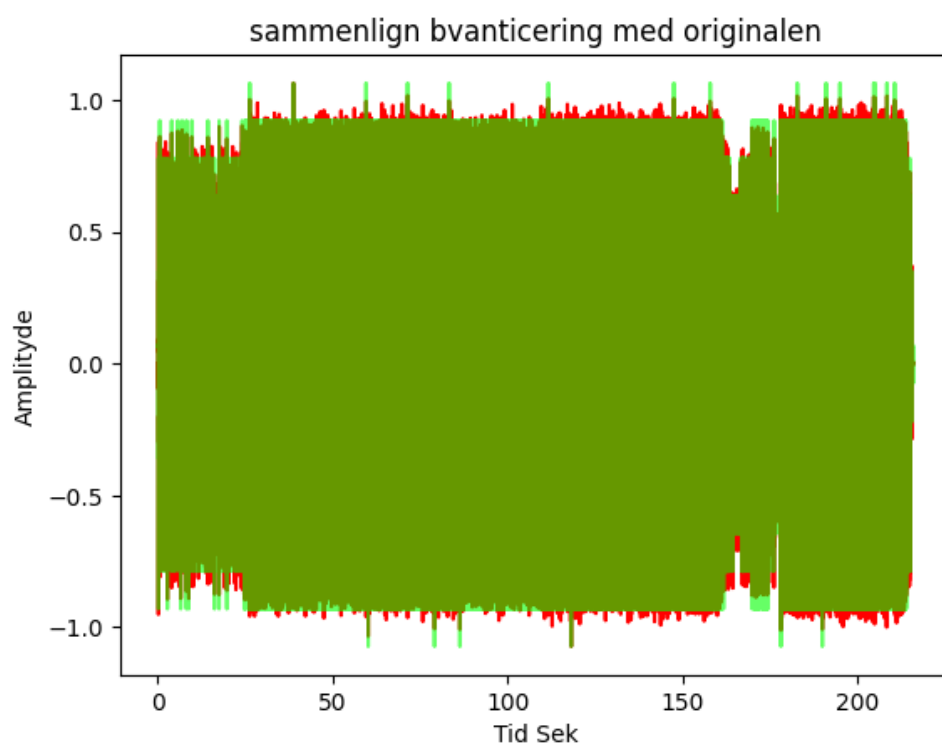
```

1 delta_max: 0.000e+00
2 delta_min: 0.000e+00

```



Figur 9: Plot af signal 2 kvanticeret med 4 bit



Figur 10: Plot af signal 2 kvantisering sammenligning

Bilag "Signal_s2_4bit.wav"

8.5 Diskussion og konklusion

Den kvantiserede musik kan stadig genkendes og høres som musik, men lydkvaliteten er blevet væsentligt forringet. Der kan høres en form for statisk støj i baggrunden af hver tone, og generelt lyder det, som om musikken er præget af meget støj og forvrængning. Lyden får dermed en karakter, der minder om det, man ofte forbinder med genren *earrape*.

9 Opgave: Fade-out på s3 med eksponentielt aftagende envelope

9.1 Opgavebeskrivelse

Lav et 'fade out' på s3 over den sidste tredjedel med en eksponentielt aftagende envelope, hvor amplituden ved slutningen er aftaget til 5%. Hint: Løs ligningen nedenfor og find hældningskoefficienten α .

9.2 Forberedelse

Her vil jeg løse ligningen i tidsdomænet, hvor (k) er tiden, hvor lyden skal fade out, og (c) er det endelige lydniveau. Jeg vil isolere (a), så den kan beregnes i Python og bruges til at fade lyden ud.

$$c = e^{-a(t)}$$

Denne formel beskriver en eksponentiel aftagning, hvor værdien ved tiden 0 er $e^0 = 1$. Aftagningen starter derfor ved 1 og falder til C , når den angivne tid t er nået. Dette kan bruges til at lave en blød aftagning af lydstyrken.

Da vi i vores tilfælde ønsker, at fade-out'et skal foregå over den sidste tredjedel af lydfilen, hvor lydniveauet skal falde til 5 %, vil værdierne være $c = 0,05$ og $t = \frac{1}{3}$ af lydfilens samlede tid.

Da vi ønsker at fade den sidste tredjedel af signalet ud, skal aftagningen først begynde ved to tredjedele af lydsignalets samlede længde. Derfor trækkes de første to tredjedele af tiden fra, så aftagningen starter ved $t = 0$ og når c præcis ved slutningen af lydfilen.

Hvis funktionen anvendes på hele signalet, vil lydfilen blive forstærket i perioden frem til to tredjedele af lydfilen. Derfor skal ligningen kun anvendes på den sidste tredjedel af lydfilen. Til dette findes indekset, hvor den sidste tredjedel begynder, så kun disse samples påvirkes af ligningen.

Afløse a:

$$c = e^{-a(t)}$$

$$\ln(c) = -a(t)$$

$$\frac{\ln(c)}{t} = -a$$

$$a = -\frac{\ln(c)}{t}$$

9.3 Kode

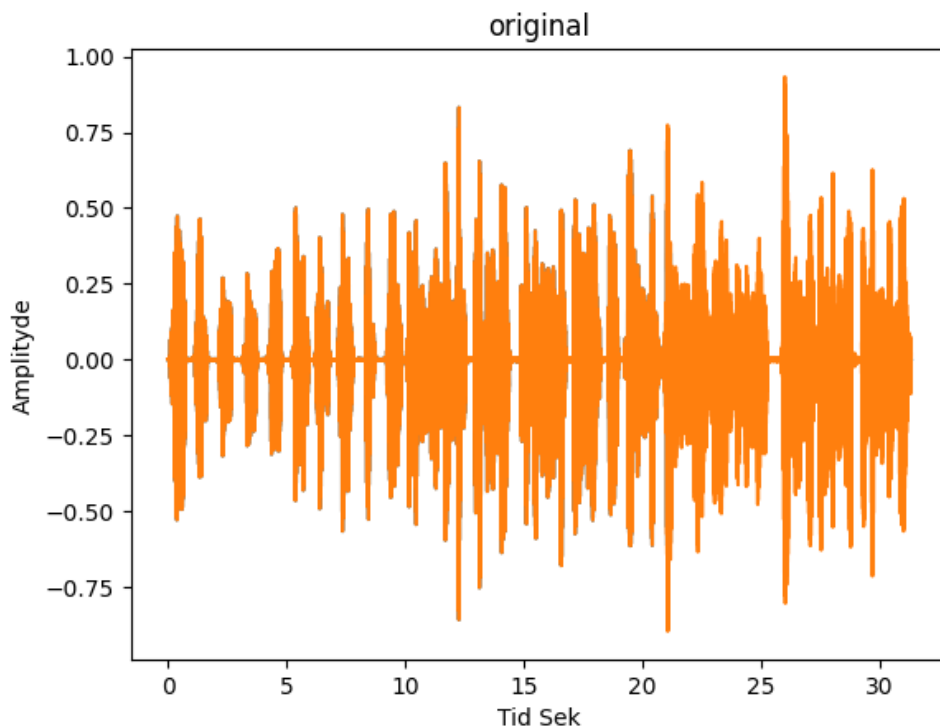
```
1 audio_s3, fs_s3 = sf.read("./Signal_s3.mp3")
2
3 #find slutningstiden af lydfilen
4 t_end_s3 = t_s3[-1]
5
6 #beregning
```

```

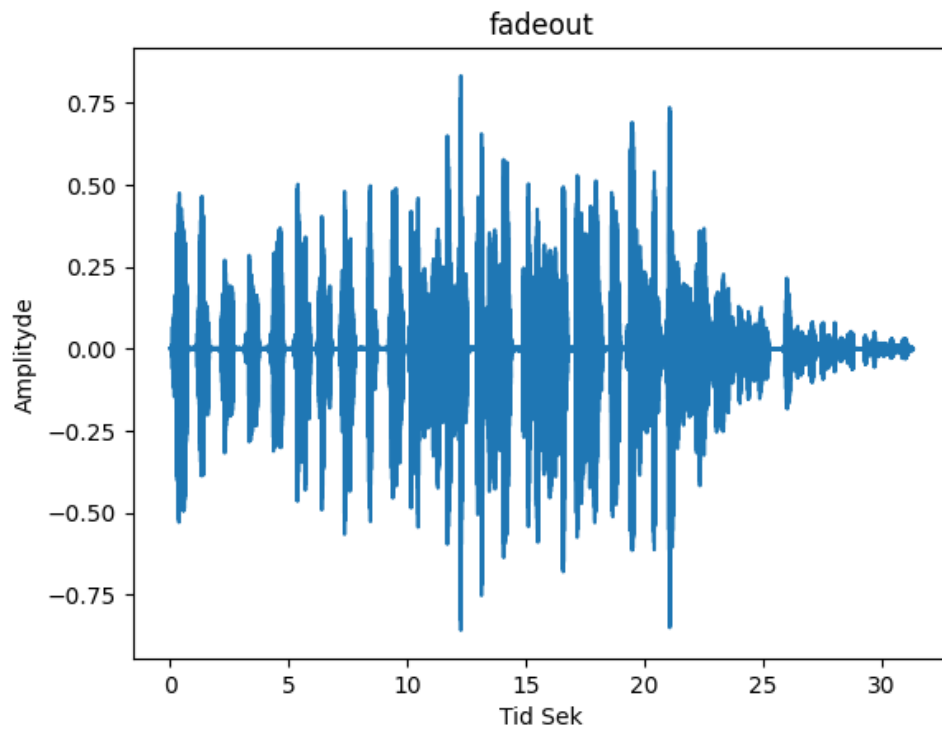
7 #c=e^(-a(t))
8 #ln(c)=-a(t)
9 #ln(c)/(t)=-a
10 #a=-ln(c)/(t)
11 a = -np.log(0.05)/(t_end_s3/3)
12
13
14 #lav ny var til fade
15 fade_s3 = 0
16 fade_s3 = audio_s3[:,0]
17
18 #find index for hvor fade skal indsættes og starte fra
19 i_2of3_s3 = (len(fade_s3)*2)//3
20
21 #lav fadeout med formelen, formelen er regnet ud fra starten 0 til 1/3 af tiden,
    derfor skal tiden hvor fadeout starter trækkes fra den aktuelle tid
22 fade_s3[i_2of3_s3:] = np.exp(-a*(t_s3[i_2of3_s3:]-t_s3[i_2of3_s3]))*fade_s3[i_2of3_s3
    :]
23
24 #plot lyden
25 plotAudio("original",t_s3,audio_s3)
26 plotAudio("fadeout",t_s3,fade_s3)
27
28 display(Audio(fade_s3,rate=f_s))

```

9.4 Resultat



Figur 11: Plot af signal 3



Figur 12: Plot af signal 3 fadeout

Bilag "Signal_s3_fadeout.wav"

9.5 Diskussion og konklusion

I plottet på Figur 12 kan vi tydeligt se, at lydfilen med fade har en reduceret amplitude i slutningen af lydfilen sammenlignet med den originale på Figur 11. Vi kan også ud fra plottet observere, at den følger den eksponentielle envelope, som vi forventer. Dette kan også tydeligt høres i lydfilen, som fader ud i den sidste tredjedel.

10 Opgave 10: Lav en ny udgave af s3, hvor der er et enkelt ekko med et delay på 100ms.

10.1 Forberedelse

For at lave echo vil jeg først kopiere lydsamplesene over i et nyt array, hvorefter jeg vil tilføje samplesene igen, men forskudt fra de oprindelige samples. Dette vil få lydsignalet til at køre to gange, men forskudt, hvilket skaber en echo-effekt. For at få det til at lyde mere som et echo vil den forskudte lyd blive reduceret med 80

For at lave offsettet vil de 100 millisekunder blive divideret med tiden mellem hvert sample. Dette giver antallet af indexes, som det ekkuede signal skal forskydes med.

10.2 Kode

```
1 #indlæs lydfilen
2 audio_s3, fs_s3 = sf.read("./Signal_s3.mp3")
3
4 #lav en var til at holde lyden med ekko
5 ekko_s3 = audio_s3[:,0].copy()
6
7 #find index offset for hvor mange indexes er mellem den aktuelle lyd til at der kommer
  ekko
8 i_ekko_100ms = (int)(0.1//t_s)
9
10 #tilføj ekko, dog dempet for at få det til at lyde som reel ekko
11 ekko_s3[i_ekko_100ms:] += audio_s3[:-i_ekko_100ms,0]*0.2
12
13
14 display(Audio(ekko_s3,rate=f_s))
```

10.3 Resultat

Bilag "Signal_s3_ekko.wav"

10.4 Diskussion og konklusion

På lydfilen kan der tydeligt høres, at lydsignalet går igennem to gange forskudt, hvor ekkosignalet anden gang er reduceret. Det lyder, som om optagelsen er foregået i et lille rungende rum med dårlig lydabsorbering.